



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich



Refactoring & Extending MILP-Based Buffer Placement

Semester Project

Yuan Qin

yuaqin@student.ethz.ch

Digital Systems and Design Automation (DYNAMO)
Computer Engineering and Networks Laboratory
ETH Zürich

Supervisors:

Jiahui Xu

Prof. Dr. Lana Josipović

May 30, 2025

Abstract

The existing buffer placement infrastructure in Dynamatic lacks both a well-structured MILP framework and support for a wide range of buffer types, limiting the integration of new buffering algorithms. To address these issues, this work implements a modularized MILP framework designed to support a wider range of algorithms, and a clearer and more flexible buffer placement system that enables the use of all buffer types. This work also resolves several issues in Dynamatic's buffering logic that previously degraded circuit performance. These changes improve the performance of existing algorithms, enable the integration of a new algorithm that aims to reduce buffer area cost while preserving optimal throughput, and provide a robust foundation for future algorithms.

Contents

Abstract	i
1 Introduction	1
2 Background	2
2.1 MILP Framework	2
2.2 Buffer Placement Pass	3
3 Issues in Existing Dynamic Buffering	5
3.1 Internal Buffers in Merge-like Components (Issue #342, PR #356)	5
3.2 Latency Imbalance on the Load-Store Queue (Issue #388, PR #356) . .	5
3.3 Memory Controller’s Load Bottleneck (Issue #401)	7
3.4 Ambiguity in the Definition of Transparent Buffer (PR #317, PR #429) .	8
3.5 FPL22’s Model Requires Correct Throughput Constraints (PR #427) . .	8
3.6 Buffering Group’s Limitations (PR #393)	9
3.7 Simplifying MILP using Precomputed Throughput (Issue #456)	10
4 Buffer Placement Pass Redesign	11
4.1 Motivation	11
4.2 Solution and Implementation (PR #317)	11
5 MILP Model Refactoring and Extension	13
5.1 Modularize MILP Constraints (PR #392)	13
5.2 Extend the MILP Structure (PR #414)	14
5.3 Add Cost-Aware Buffers Algorithm (PR #472)	14
6 Conclusion	15
Bibliography	16

1 Introduction

We previously proposed a new buffer placement algorithm that achieves lower area cost than existing algorithms while preserving throughput optimization. This algorithm leverages a broader set of buffer types and aims to minimize area by preferentially selecting more area-efficient buffer implementations whenever feasible.

The algorithm was evaluated in a custom Python-based framework, and the results validated its effectiveness. However, several challenges remain:

- The algorithm has not yet been integrated into the MLIR-based Dynamatic framework.
- Dynamatic currently lacks support for a wide range of buffer types and flexible MILP modeling, both of which are essential for the algorithm.
- Several existing issues in Dynamatic alter the intended modeling of both existing and new buffer placement algorithms, resulting in degraded circuit performance. Resolving these issues is essential to accurately evaluate the new algorithm’s performance.

To address these limitations, we introduce the following improvements:

- Extend the buffer description to support more flexible buffer types.
- Introduce modular MILP constraint functions for easier extensibility and customization.
- Resolve existing issues in Dynamatic that impact both the existing and new algorithms.
- Integrate the new buffer placement algorithm into the MLIR-based Dynamatic framework.

The remainder of this document is structured as follows. Section 2 provides background on how buffer placement is handled in Dynamatic, including the MILP framework and the buffer placement pass. Section 3 identifies existing issues in Dynamatic that affect the modeling and performance. Section 4 describes our modifications to the buffer operation and placement logic. Section 5 details the refactoring and extension of the MILP model and the integration of our proposed algorithm. Finally, Section 6 concludes the work.

2 Background

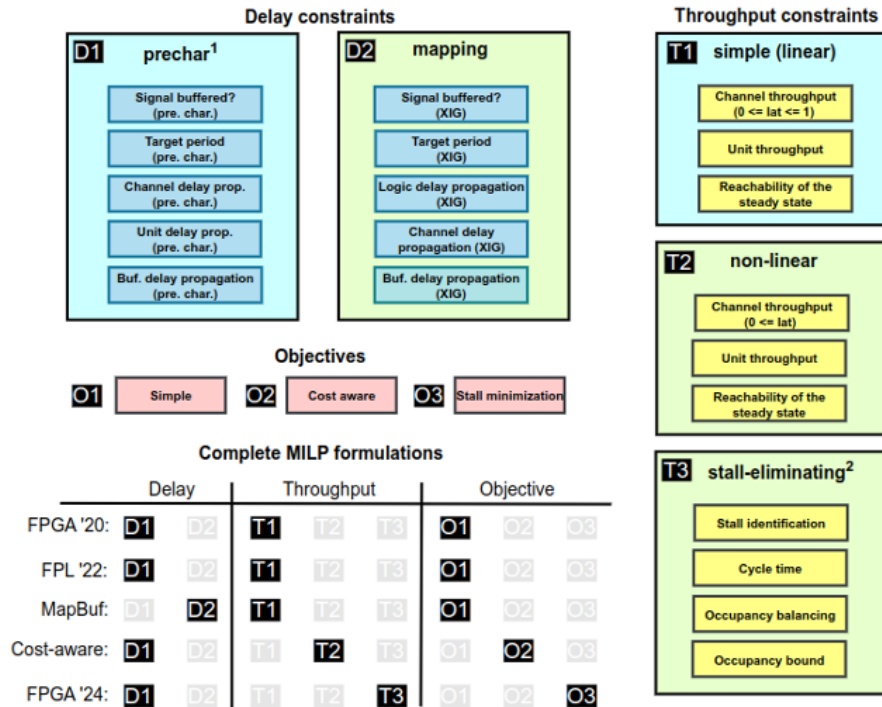
This section outlines how buffering is handled in the previous version of Dynamatic, which consists of two main phases: the MILP-based buffer placement algorithm and the buffer placement pass.

Given an unbuffered dataflow circuit, the MILP model is used to determine a buffer placement strategy that maximizes circuit throughput. The solution specifies whether a buffer should be placed on each channel and provides attributes for those buffers.

After the MILP optimization, the buffer placement pass extracts the MILP results and converts them into specific attributes. These attributes are written into a JSON configuration file, which guides the RTL backend in instantiating the appropriate buffer HDL modules.

The result is a buffered circuit that can be simulated and synthesized. The remainder of this section discusses the MILP framework and the buffer placement pass in more detail.

2.1 MILP Framework



¹The channel/unit delay propagation constraints can account for any subset of {data, valid, ready, spec, tag}.

²Some constraints are omitted for brevity; in the paper, this formulation was separated into 2 MILPs for linearization

Figure 1: An Ideal MILP Framework

Dynamatic provides a base class, `BufferPlacementMILP`, which defines the core infrastructure for MILP-based buffer placement algorithms. It offers a reusable interface for adding variables,

constraints, and objectives to the optimization model. Specific MILP algorithms are implemented in derived classes such as `FPGA20Buffers` [1] and `FPL22Buffers` [2]. These derived classes construct their models by providing customized input parameters and selectively invoking functions from the base class.

As more MILP algorithms are developed, the existing variables and functions in the base class are no longer sufficient to capture the new features required by these algorithms. Therefore, it is necessary to extend the base class to support a broader range of algorithms in a unified and reusable way.

Figure 1, proposed in [Issue #228](#), presents an ideal framework for organizing the new base class and illustrating how different derived classes can build upon it. As shown in the figure, the base class is structured around three components:

- **Timing constraints (denoted by Delay):** These constraints ensure correct timing behavior by analyzing the delay of each unit and channel, and by placing buffers that break timing paths to satisfy clock cycle boundaries. In the figure, two boxes under this category represent fundamentally different formulations for the same timing property.
- **Throughput constraints:** These constraints restrict the achievable throughput of the circuit by analyzing latencies introduced by buffers and units. The figure includes three boxes, each representing a distinct modeling approach.
- **Objective:** The primary goal is to maximize throughput; the secondary goal is to minimize buffer usage. There are three alternative objectives in the figure.

Each derived class (i.e., each specific algorithm) selects one timing constraint box, one throughput constraint box, and one objective function that best suit its characteristics. The bottom-left corner of the figure represents five algorithms and the specific constraint and objective modules each one selects.

This modular structure enables a clear separation of different modeling strategies, allows each algorithm to customize its constraints and objective, and promotes code reuse through shared base class functions.

Previous to this work, only the D1, T1, and O1 modules shown in the figure are implemented in the base class, along with two derived classes: `FPGA20Buffers` and `FPL22Buffers`.

2.2 Buffer Placement Pass

Module Name	Timing Attribute	Slot Number
OEHB	Opaque	1
TEHB	Transparent	1
OFIFO	Opaque	>1
TFIFO	Transparent	>1

Table 1: Buffer HDL modules

The buffer placement pass interprets the MILP solution and generates corresponding `BufferOp` operations in the IR. In previous versions, buffers are categorized into two types: *Opaque* and

Transparent. For each channel, the pass assigns the number of Opaque and Transparent slots based on the MILP output.

Opaque buffers hold a token for at least one cycle before forwarding it. In contrast, Transparent buffers may forward tokens in the same cycle if the downstream is ready.

Each `BufferOp` encodes two attributes: the slot number, which defines the buffer’s token capacity, and a timing attribute that specifies which of the three handshake signal paths—`data`, `valid`, and `ready`—are broken. Opaque buffers break the `data` and `valid` paths, while Transparent buffers break the `ready` path.

At the RTL backend, this information is written to a JSON file, which guides the instantiation of specific HDL modules. Table 1 lists the available modules and the corresponding attribute combinations used for selection.

This infrastructure serves as the basis for further enhancements, including support for more diverse buffer types and a more modular and extensible MILP formulation.

3 Issues in Existing Dynamic Buffering

While Dynamic’s existing buffer placement framework provides a solid foundation, several issues within its current implementation hinder optimal performance and accurate modeling. These issues, if unaddressed, can compromise the effectiveness of new buffer placement algorithms and lead to unfair comparisons with existing ones. Addressing them not only improves the modeling fidelity and performance of new MILP formulations, but also enhances the original implementations. This section outlines several key problems and presents the corresponding solutions where available.

3.1 Internal Buffers in Merge-like Components ([Issue #342](#), [PR #356](#))

Problem

Certain control-related components in Dynamic, namely **Merge** and **Mux**, previously included an internal buffer on their output ports in the RTL design. These buffers were originally introduced to avoid token reordering issues (see [Issue #342](#)) when such components lie on a cycle and are followed by a **Fork**—a pattern known to cause functional mismatches if unbuffered.

However, these buffers are not part of the components’ core semantics and introduce two main issues:

- **Redundant area cost:** Each instance unconditionally adds a buffer, increasing circuit area even when no additional buffering is required.
- **Modeling complexity:** The MILP and buffer placement pass must include ad hoc logic to account for internal buffers not reflected in the IR.

Solution

The internal buffers in **Merge** and **Mux** have been removed from the RTL. Instead, the MILP model enforces additional constraints to prevent token reordering when necessary. In many cases, no extra buffer is needed, as the MILP may already place one after a merge-like component for throughput or other optimization goals. The buffer placement pass strictly follows the MILP solution and no longer considers internal RTL buffers.

This change was implemented in [PR #356](#), with supporting cleanup completed in [PR #358](#).

3.2 Latency Imbalance on the Load-Store Queue ([Issue #388](#), [PR #356](#))

Problem

The problem is described in [Issue #388](#). Figure 2a illustrates a typical circuit structure involving an LSQ. In Dynamic, the LSQ manages memory operations by coordinating store and load accesses through group-based allocation, which is typically triggered by a **ControlMerge**. The **Store** operation transmits its address and data to LSQ only after the allocation has been triggered.

However, this design introduces an asymmetric latency between the two inputs of the LSQ. Specifically, the **Store**-side token arrives one cycle later than the token from the **ControlMerge**, resulting in a higher latency on the **Store** path.

Since the **Store** path lacks buffering, the mismatch causes the back pressure to propagate upstream, which degrades circuit throughput.

In addition to **Store** operations, **Load** operations are also affected by the same issue.

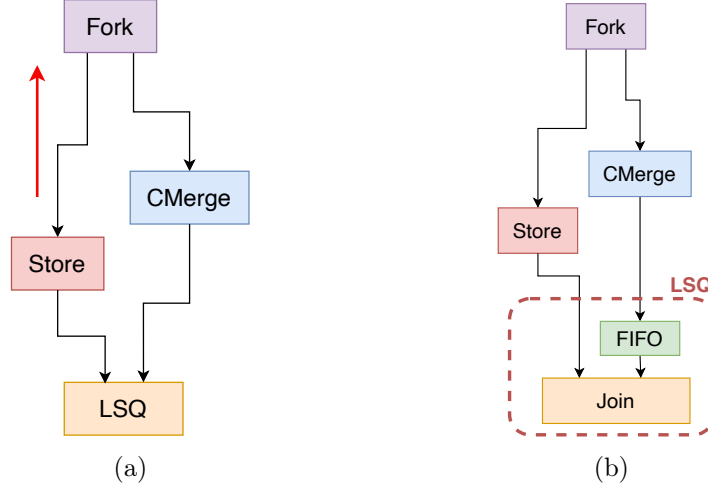


Figure 2: Structure Involving LSQ and Equivalent Throughput Model

Solution

An Unrealized Solution:

We constructed a throughput model for the circuit structure shown in Figure 2a. In this structure, the LSQ can receive any number of allocation tokens before a **Store** operation. However, each **Store** could send its token to the LSQ one cycle after the corresponding allocation. Furthermore, **Store** operations can only proceed one cycle after the previous **Store** operation.

Focusing on the reconvergent paths of allocation and **Store** shown in Figure 2a, we model the LSQ as a join, with a FIFO placed on the allocation side, as illustrated in Figure 2b. A token entering the FIFO from the allocation path is considered to have entered the LSQ, while a token entering the join from the **Store** path is considered to have entered the LSQ. The FIFO has a one-cycle latency and unlimited capacity, allowing allocation tokens to be consumed by the LSQ unconditionally. However, a **Store** token can enter the join only when its corresponding allocation token has exited the FIFO, that is, one cycle after the allocation token entered it.

The main limitation of this model is that it only captures the interaction between this specific pair of reconvergent paths. In practice, the LSQ interacts with many other paths, and this simplified model does not account for those relationships.

In our implementation, we used a temporary workaround by enforcing buffers on the upstream channels of load and store operations. The buffers mitigate or solve the backpressure on those paths. Experimental results show that this approach leads to a notable improvement in

throughput.

3.3 Memory Controller's Load Bottleneck ([Issue #401](#))

Problem

The buffer placement MILPs in Dynamatic assume that multiple Load operations connected to the same **MemoryController** can access the RAM concurrently. However, this assumption breaks down when these loads belong to the same CFDFC and the RAM has only a single read port.

Figure 3 illustrates such a case, where two Load operations within the same CFDFC share a single **MemoryController**. The MILP estimates the best initiation interval (II) of 1 as shown in Figure 4a, assuming parallel access. In practice, due to port contention, the actual II increases to 2, as shown in Figure 4b. This throughput overestimation can lead to suboptimal buffer placement decisions and degraded circuit performance.

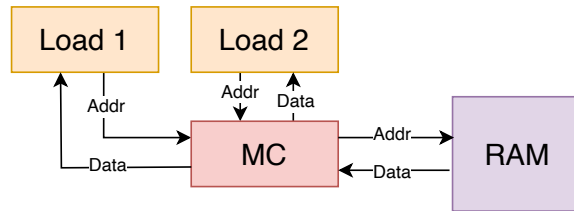


Figure 3: Circuit with two load units connected to the same Memory Controller

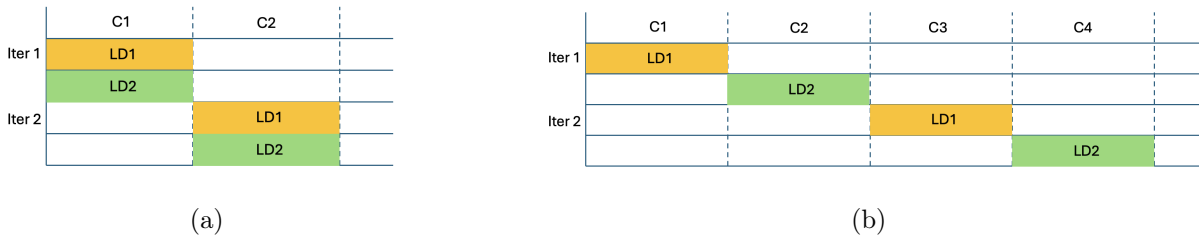


Figure 4: Initiation Interval of Load Operations

Solution

A possible solution is to statically analyze each CFDFC. If a CFDFC contains at most N Load operations connected to the same **MemoryController**, the throughput upper bound for that CFDFC should be limited to $1/N$ to reflect the port sharing constraint.

However, after analyzing 83 benchmarks, no CFDFC was found to contain multiple Load operations targeting the same memory controller. As a result, no solution has been implemented at this time. The issue remains open and will be revisited if such cases are encountered in future designs.

3.4 Ambiguity in the Definition of Transparent Buffer (PR #317, PR #429)

Problem

In the previous buffer placement pass, buffers were categorized as *opaque* or *transparent*. By definition, in the timing context, an opaque buffer breaks the data and valid paths, while a ready buffer breaks the ready path. However, the actual behavior of buffers—particularly with respect to the ready path—does not fully align with this definition. Among the HDL modules representing transparent buffers, **TEHB** breaks the ready path, whereas **TFIFO** does not break any path. Another example is **OFIFO**, which, as an opaque buffer, breaks not only the data and valid paths but also the ready paths.

This ambiguity impacts both of the existing two MILP models. The FPGA20 model assumes that transparent buffers do not break any signal paths, which applies only to **TFIFO**. In contrast, the FPL22 model assumes that transparent buffers break the ready path, thereby requiring a **TEHB**, which cannot be substituted by a **TFIFO**. However, both models arbitrarily use **TEHB** and **TFIFO** to represent transparent buffers, as shown in Table 1, rendering each model incorrect in certain scenarios.

Solution

We remove the use of the terms *opaque* and *transparent* in the buffer placement pass. Instead, we introduce explicit timing attributes to describe the buffer’s behavior: **break_dv**, **break_r**, **break_dvr**, and **break_none**, covering all four cases. Under this convention, **TEHB** corresponds to **break_r**, and **TFIFO** corresponds to **break_none**.

This change allows MILP models to consistently use HDL modules according to their actual timing attributes.

3.5 FPL22’s Model Requires Correct Throughput Constraints (PR #427)

Problem

The original throughput constraints in the buffer placement MILP were based on FPGA20’s assumption that buffers break all three signals—data, valid, and ready (DVR)—simultaneously. This behavior was encoded through a single boolean variable $R_c = \text{dataBuf} \wedge \text{readyBuf}$, where **dataBuf** indicates the buffer breaks the data/valid path, and **readyBuf** indicates it breaks the ready path. When R_c holds, the throughput constraints enforce that both token occupancy (due to breaking DV) and bubble occupancy (due to breaking R) must be greater than or equal to the CFDFC’s throughput.

However, FPL22’s buffer model allows **dataBuf** and **readyBuf** to be set independently, enabling four distinct configurations: breaking no path, only DV, only R, or both. The original throughput constraints could not express these configurations. To address this, we decompose R_c into two independent conditions:

- If **dataBuf** is true, then token occupancy $\geq$ CFDFC’s throughput
- If **readyBuf** is true, then bubble occupancy $\geq$ CFDFC’s throughput

This decomposition reveals a limitation in the FPGA20’s buffer model: it does not explicitly model ready paths, preventing the MILP from placing R-breaking buffers. This can leave combinational cycles on the ready path unresolved.

Solution

We revised the MILP formulation to support independent `dataBuf` and `readyBuf` properties, enabling an accurate throughput model for FPL22.

To resolve the lack of ready path modeling in FPGA20, we place additional TEHBs at the output channels of all Merge/Mux units on combinational cycles. Since TEHBs break ready paths while maintaining throughput unaffected, they resolve this issue without performance degradation. This workaround is specific to FPGA20’s model and does not affect other buffer algorithms.

3.6 Buffering Group’s Limitations (PR #393)

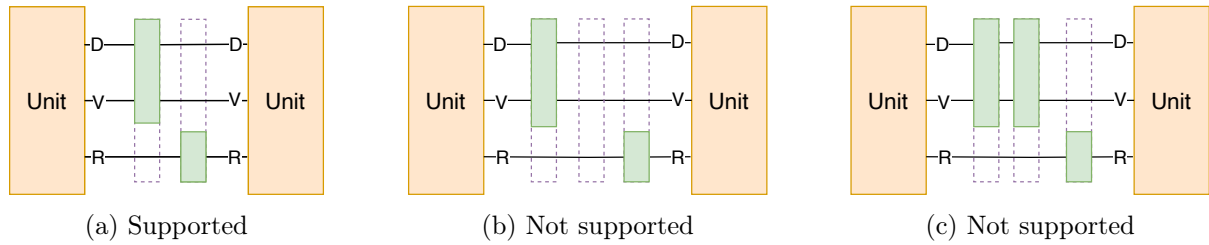


Figure 5: Buffering Group

The buffering group abstraction was introduced to support FPL22’s timing model, with future extensibility in mind. As shown in Figure 5a, it defines two groups on each channel: one before and one after. In the figure, a green box indicates that the buffer breaks the path, while a dashed box indicates the buffer does not break the path, only introducing a combinational delay. In FPL22, the “before” group breaks the data and valid paths, and the “after” group breaks the ready path.

The buffering group is mainly used to model the I/O port delay of the buffer when a signal path is broken, and the combinational delay of the buffer when it is not. It also enforces a minimum slot number on the channel to ensure that there are enough slots to accommodate all groups.

However, this abstraction has limited practical use, especially as more buffer types are introduced:

- The timing model assumes exactly two groups. If a channel has three or more groups, as shown in Figure 5b, the model cannot represent them.
- If a buffer does not break any path, like the one in the middle of Figure 5b, the model cannot distinguish between the absence of a buffer and a buffer with combinational delay on all paths.

- The timing model does not consider the slot number of each group. For example, in Figure 5c, if the group that breaks data and valid contains two concatenated OEHB slots, the combinational delay on the ready path should double. The buffering group abstraction has no way to express this.

The buffering group is not essential for FPL22 and can be safely removed or kept if desired. However, due to these limitations, it cannot be applied to the latest buffer model.

3.7 Simplifying MILP using Precomputed Throughput ([Issue #456](#))

For MILP algorithms that do not model buffer delay, solving time can be significantly reduced by precomputing the optimal throughput.

The MILP constraints are divided into two categories: timing constraints and throughput constraints. Timing constraints force buffer placement on certain channels to break specific signal paths. The maximum achievable throughput of a CFDFC is determined by the reciprocal of the maximum number of data and valid path breaks in any cycle. If we apply timing constraints only to data and valid paths, this upper bound can be computed directly.

When buffer delay is not modeled, it is always possible to place additional ready-breaking buffers to satisfy timing constraints on ready paths. These buffers do not affect throughput. Similarly, throughput constraints are primarily used to control the slot number of buffers to ensure latency equivalence across reconvergent paths. We can place any number of `break_none` buffers on non-cyclic channels to balance latency without affecting throughput. If the MILP algorithm does not model buffer delay, all additional buffers do not interfere with the timing constraints previously applied.

Therefore, a valid buffer placement always exists that satisfies all constraints while achieving the maximum throughput computed using only data and valid timing. Based on this, we adopt a two-stage optimization strategy. In the first stage, we solve for the optimal throughput using only data and valid timing constraints. In the second stage, this throughput value is treated as a constant and substituted into the full MILP model. As a result, linear terms become constants, and quadratic terms become linear, which greatly reduces model complexity.

This strategy is applied to the new MILP algorithm involving quadratic constraints and has led to speedups of at least several hundred times on certain benchmarks.

4 Buffer Placement Pass Redesign

4.1 Motivation

The previous buffer placement pass used the *timing attribute* as the sole criterion for distinguishing buffer types. However, this approach fails to capture other essential aspects of buffer behavior, such as throughput (elasticity) and area cost. As a result, multiple buffer types with identical timing characteristics were treated as interchangeable. Recent MILP models, however, make use of new buffer types that reflect trade-offs between elasticity and area, making it necessary to distinguish buffers not only by timing but also by other properties.

In particular, the three buffer types listed in Table 2 that break the **data** and **valid** paths (i.e., **break_dv**) differ in both elasticity and area cost:

- **FIFO_BREAK_DV**: Equivalent to an **OFIFO** without the **TEHB**. This type has high elasticity, which benefits throughput, but its area cost is high and the extra elasticity is often unnecessary.
- **ONE_SLOT_BREAK_DV**: Previously known as **OEHB**, this buffer type now supports chaining to achieve higher token capacity. The slot number corresponds to the number of **OEHB**s in the chain. It provides a low-area-cost alternative to **FIFO_BREAK_DV**, at the expense of reduced elasticity.
- **SHIFT_REG_BREAK_DV**: Introduced in [PR #407](#), this buffer behaves similarly to a delay buffer concatenated with an **OEHB** in arithmetic units, where all slots either forward data simultaneously or hold together. It has very low elasticity but minimal area cost.

Type Name	Associated	Elasticity	Area Cost
FIFO_BREAK_DV	OFIFO	High	High
ONE_SLOT_BREAK_DV	OEHB	Low	Low
SHIFT_REG_BREAK_DV	Delay buffer	Very low	Very low

Table 2: Three distinct buffer types breaking DV

We need a new classification method that not only distinguishes the three buffer types discussed above, but also accommodates potential buffer types that may be introduced in the future.

4.2 Solution and Implementation ([PR #317](#))

To address the information loss of the *timing attribute*, two solutions are possible. One is to characterize each buffer by combining several orthogonal attributes. The other is to assign an explicit *buffer type* attribute that directly identifies the corresponding HDL module.

We favor using explicit buffer types over extending attribute-based classification, for the following reasons:

- **Sparsity of the attribute space:** Although it is possible to distinguish the three DV-breaking buffer types by introducing two additional attributes, those attributes are irrelevant for most other buffer types. For example, using “shift register” as an attribute only makes sense for DV buffers—a shift-register form of R-breaking or DVR-breaking buffers would only degrade performance and are not considered. As a result, the attribute space becomes sparse, with many unused combinations, which complicates maintenance and adds modeling overhead without improving generality.
- **Low attribute utilization:** Apart from timing, no other attribute is widely recognized or consistently used across different passes or MILP models. Such attributes lose practical value.
- **Limited extensibility:** If a new buffer type introduces a behavior not captured by existing attributes, the current scheme fails to represent it accurately.
- **Limited MILP flexibility:** Different MILP models may use different logic to select buffers. Describing buffers using attribute combinations imposes a fixed interpretation in the RTL backend, forcing all MILPs to follow the same rule and limiting their flexibility. In contrast, buffer types give each MILP full control and are directly translated by the backend.

We implemented the following changes to enable use of the buffer type attribute in the buffer placement pass:

- Extended `handshake::BufferOp` with a new `buffer type` attribute.
- Updated the JSON used for hardware generation to select HDL modules based on the `buffer type` attribute, instead of relying on a combination of timing attribute and slot number.
- Each buffer type is associated with a configurable slot number, and a channel can place any combination of buffer types, enabling maximum flexibility in buffer placement to support diverse MILP strategies.
- Retained the original timing attribute in `BufferOp`, as it remains relevant for analyses and decisions outside the buffer placement pass.

This redesign provides a foundation for future MILP models that require finer-grained buffer placement strategies based on more than just timing.

5 MILP Model Refactoring and Extension

In the background section, Figure 1 introduced a modular framework for MILP-based buffer placement. Each timing or throughput module is shown as a separate box in the figure. The original implementation in `Dynatomic` only supports the D1, T1, and O1 modules, as used by the `FPGA20Buffers` and `FPL22Buffers` algorithms. In this project, we implement the T2 and O2 modules, along with a new algorithm named `CostAwareBuffers`, which leverages T2 and O2.

Before implementing T2 and O2, we first modularize the constraint functions. In the original codebase, a single function often mixes several logically independent constraints. Some of these constraints are basic and appear in multiple boxes in Figure 1. If not separated, they must be redundantly written in each box-specific function. Modularization not only avoids such duplication but also clarifies each constraint’s purpose and simplifies maintenance.

This chapter first introduces the modularization of constraint functions, then describes the extension of the base class that implements the T2 and O2 modules, and finally presents the design of the newly implemented `CostAwareBuffers` algorithm.

5.1 Modularize MILP Constraints (PR #392)

We introduce the following constraint components to structure the MILP model in a more modular and maintainable way. Each component captures a specific aspect of circuit behavior and can be selectively reused across different algorithms.

- **Channel Timing:** Enforces timing constraints on each signal (data, valid, ready) along a channel, modeling how buffers on the channel affect the timing propagation.
- **Unit Timing:** Describes timing propagation on each signal (data, valid, ready) across units.
- **Buffer Presence:** Specifies basic buffer properties. A buffer is considered present if it has a positive number of slots. If any signal has a buffer, the entire channel is treated as buffered.
- **Buffering Group:** Defines constraints for grouped signals. Ensures consistent buffer placement across related signals (e.g., data and valid) and that the slot number is sufficient to accommodate all distinct buffering groups.
- **Steady-State Reachability:** Ensures that the computed throughput corresponds to a reachable steady-state.
- **Channel Throughput:** Models how buffer latency and token occupancy on a channel constrain the circuit throughput.
- **Unit Throughput:** Models how the latency of pipelined units influences the circuit throughput.

This modular structure enables constraint reuse across different MILP algorithms and lays the foundation for the structural extensions introduced in the next section.

5.2 Extend the MILP Structure (PR #414)

We implement the T2 and O2 modules introduced in Figure 1 to support the new algorithm **CostAwareBuffers**, which requires features not covered by the earlier T1 and O1 modules. This involves extending the MILP with new variables, constraints, and an objective function.

T2 makes the throughput model more detailed by replacing the boolean **dataBuf** variable with an integer-valued **dataLatency**, which captures not only whether the buffer breaks the DV path but also how many cycles of latency it introduces. An additional variable, **shiftReg**, indicates whether the buffer type **SHIFT_REG_BREAKING_DV** is used, which triggers stricter throughput constraints.

Compared to O1, which minimizes buffer usage by reducing the total number of buffer slots, O2 incorporates the area cost of each buffer type. Each type is assigned a penalty based on its approximate LUT usage, estimated from synthesizing the buffer in isolation. This allows the MILP to prefer area-efficient buffer types once throughput requirements are met.

5.3 Add Cost-Aware Buffers Algorithm (PR #472)

We implement a new algorithm, **CostAwareBuffers**, which combines the D1, T2, and O2 modules. It enables the placement of more area-efficient buffer types while maintaining maximum throughput through T2. And O2 guides buffer type selection toward lower LUT usage.

These features introduce two levels of trade-off. At the buffer level, more area-efficient buffers are subject to stricter throughput constraints. At the MILP level, a wider range of buffer choices, more detailed constraints, and a more refined objective increase model complexity and solving time.

6 Conclusion

This work enables the integration of the **CostAwareBuffers** algorithm into Dynamatic by resolving key issues in existing buffer modeling, redesigning the buffer placement pass to support clearer and more flexible instantiation, and extending the MILP framework to accommodate a broader range of algorithms. These improvements not only enhance the performance of current algorithms but also provide a robust foundation for future algorithms.

Bibliography

- [1] L. Josipović, S. Sheikhha, A. Guerrieri, P. Ienne, and J. Cortadella, “Buffer placement and sizing for high-performance dataflow circuits,” in *Proceedings of the 28th ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, Seaside, CA, Feb. 2020, pp. 186–96.
- [2] C. Rizzi, A. Guerrieri, P. Ienne, and L. Josipović, “A comprehensive timing model for accurate frequency tuning in dataflow circuits,” in *Proceedings of the 22nd International Conference on Field-Programmable Logic and Applications*, Belfast, UK, Aug. 2022.